Production Launch Fix Guide

A step-by-step remediation process for every requirement marked MISSING in the Complete requirements inventory, grouped by the same sections and in the order they should be worked.

Project	resume-co (Resumewriter.ai)
Branch	handover/production-readiness
Source	production-launch-readiness-audit.html, Complete requirements inventory
Scope	75 requirements marked MISSING across 12 sections
Audience	the engineer taking this to production
Dated	28 October 2026

READ THIS FIRST

This guide covers the **MISSING** rows only. It is not the whole job. Sixteen further requirements are marked BLOCKED in the inventory because they need access nobody has used yet (production environment, the live database and its backups, the Stripe and Resend dashboards, repository settings). Those cannot be inferred from code and are not covered here.

Completing every step in this guide takes the product from *unsafe to launch* to *ready to be verified*. The verification itself - running the journeys against staging, checking the blocked items with real access, and confirming the live site after deploy - is what earns a go-live.

WORKING ORDER

Sections are numbered as in the inventory, but do not work them strictly top to bottom. Three sequencing rules matter more than the numbering:

1. **1.** Fix the lint gate (section 11) **before any feature work**. It touches up to 85 files and will conflict with every open branch if left until later.
2. **2.** Do the migration ledger (section 5) early. Sections 2, 4 and 12 all need to add a table, and there is no safe way to do that until it exists.
3. **3.** Set `allowDangerousEmailAccountLinking: false` (section 2) today. It is one line and closes the only account-takeover path in the audit.

Within each section, items are already ordered so that shared files are touched once. Where an item depends on another section, it says so.

Contents

Section	Missing	Items covered
1. Pages and application shell	8	6
2. Authentication and account lifecycle	6	6
3. Email and notifications	7	6
4. Payments, subscription and entitlement	10	7
5. Data, database and storage	9	7
6. Privacy and compliance	7	5
7. Security	5	5
8. SEO, discovery and advertising	5	5
9. UX, mobile and accessibility	2	2
10. Testing	5	5
11. Build, CI/CD and environments	4	4
12. Monitoring and operations	7	4

Item counts differ from the missing counts where tightly related requirements are handled as one piece of work - the six missing emails in section 3, for example, share one sender and one helper.

1. Pages and application shell

8 requirements marked missing. Eight gaps. Three are error-handling files Next.js expects you to provide and will fall back to bare defaults without. Five are pages the product promises implicitly but never ships. Start here: these are the cheapest fixes on the whole list and two of them change what a user sees when something breaks.

Custom 404 page, error boundary and loading states

task R1

WHY IT MATTERS

There is no not-found.tsx, error.tsx, global-error.tsx or loading.tsx anywhere in src/app. Today an unhandled render error in production shows the bare Next.js error screen, a bad URL shows the stock 404, and slow routes show nothing at all. These are the three files every App Router project is expected to supply, and their absence is most visible exactly when a user is already having a bad time.

STEPS

1. Create src/app/not-found.tsx. Give it the real app chrome, a short human sentence (not 'Error 404'), and one primary action back to / for signed-out users or /dashboard for signed-in ones.
2. Create src/app/error.tsx as a client component ('use client'). It receives { error, reset }. Show a plain apology, a Try again button wired to reset(), and a link out. Never render error.message to the user: it can leak internals.
3. Create src/app/global-error.tsx for failures in the root layout itself. It must render its own <html> and <body> because the layout failed. Keep it dependency-free, inline-styled, with no imports from your component library.
4. Add loading.tsx for the slow routes first: /dashboard, /jobs, /tailoring. Use a skeleton matching the final layout, not a bare spinner, so the page does not jump when content lands.
5. Report the error to Sentry from error.tsx with captureException(error) inside a useEffect, so these failures reach your dashboard rather than dying in the browser.

FILES

`src/app/{not-found,error,global-error}.tsx, plus loading.tsx per route segment`

DONE WHEN

Visiting a nonsense URL shows your 404 with working navigation. A deliberately thrown error in a page shows your error UI, offers a working retry, and appears in Sentry. No raw stack trace is ever visible to a user.

About page

task M8

WHY IT MATTERS

Only four public routes exist: /, /login, /privacy, /terms. There is no page explaining who operates this service. That matters commercially (trust before someone uploads a resume) and practically: payment processors and ad networks both look for a real identity behind a paid product.

STEPS

1. Create src/app/(public)/about/page.tsx with real content: who runs the product, where the company is based, and what the AI does and does not do with a user's resume.
2. Export page metadata (title, description) so it is indexable, following the pattern already used in the root layout.
3. Add it to the sitemap in src/app/sitemap.ts, priority around 0.5.
4. Link it from the footer alongside the existing legal links.
5. Do not use placeholder copy. An About page with lorem text is worse than none, and the QA handbook explicitly checks for placeholder text.

FILES

`src/app/(public)/about/page.tsx, src/app/sitemap.ts, footer component`

DONE WHEN

The page loads signed out, is listed in the sitemap, is linked from the footer, and names a real operating entity.

Contact and support page

task M8

WHY IT MATTERS

There is no support route and no contact address anywhere in the app. A paying customer with a billing problem currently has no route to you. This is also a hard requirement for Stripe (customers must be able to reach the merchant) and for the privacy contact obligation covered in section 6.

STEPS

1. Create src/app/(public)/contact/page.tsx. The simplest honest version is a monitored email address plus expected response time. A form is optional.
2. If you build a form, add POST /api/contact with Zod validation and a rate limit, reusing the limiter in src/permissions/rate-limit.ts. Do not ship an unthrottled form endpoint.
3. Make the same address serve as the privacy contact so section 6 is satisfied by the same work.
4. Link it from the footer and from the error and 404 pages built above.
5. Add it to the sitemap.

FILES

`src/app/(public)/contact/page.tsx, optional src/app/api/contact/route.ts`

DONE WHEN

A signed-out visitor can find a working way to reach you within two clicks of any page, and a test message is received.

Public pricing page

task M8

WHY IT MATTERS

/payment sits behind authentication, so nobody can see what the product costs before creating an account. That suppresses conversion and, combined with the price mismatch in section 4, means the first time anyone sees a price is at checkout.

STEPS

1. Create src/app/(public)/pricing/page.tsx rendering the same plans from src/services/payments/plans.ts that checkout uses. Do not retype the figures.
2. Fix the price mismatch first (section 4, task P4). Publishing a public page with the wrong price makes the disclosure problem worse, not better.
3. State plainly what is charged today and what renews later, including the trial terms and the renewal amount.
4. Link the primary action to /login with a next parameter so the user returns to checkout after signing in.
5. Add it to the sitemap and the footer.

FILES

`src/app/(public)/pricing/page.tsx, src/app/sitemap.ts`

DONE WHEN

Prices are visible signed out, come from the plan objects rather than hardcoded strings, and match what Stripe actually charges.

Refund and cancellation policy

task M8

WHY IT MATTERS

Neither /terms nor any other page states a refund or cancellation policy. For a subscription with a trial that converts to a recurring charge, this is a consumer-law requirement in the EU, UK and several US states, and a common cause of Stripe account review.

STEPS

1. Decide the actual policy with the business: refund window, whether trial conversion is refundable, and how a cancellation takes effect (immediately or at period end).
2. Write it into /terms as its own clearly titled section, or as a dedicated /refund-policy page.
3. Make the wording match what the code does. If cancelling leaves access until period end, say so.
4. Link it from checkout, so it is visible before payment rather than after.
5. Pair this with the self-serve cancellation work in section 4. A stated policy a user cannot act on is still a complaint.

FILES

`src/app/(public)/terms/page.tsx or a new refund policy route`

DONE WHEN

A customer can read, before paying, how to cancel and whether they can get money back, and the stated behaviour matches the implementation.

Cookie policy page

task M3

WHY IT MATTERS

PostHog and Sentry load unconditionally, so the product sets analytics storage without disclosure or consent. There is no cookie policy page to disclose it on. This is the documentation half of the consent work in section 6.

STEPS

1. List every client-side storage the app uses: the Auth.js session cookie, the resume-co: localStorage namespace, PostHog, and Sentry.
2. Write src/app/(public)/cookies/page.tsx naming each one, its purpose, and whether it is strictly necessary or optional.
3. Classify honestly: the session cookie is necessary, analytics are not.
4. Link it from the consent banner you build in section 6, and from the footer.
5. Keep it aligned with the privacy policy rewrite in section 6 so the two do not contradict each other.

FILES

`src/app/(public)/cookies/page.tsx`

DONE WHEN

Every storage mechanism the app uses is named on the page, and the classification matches what the consent gate actually enforces.

2. Authentication and account lifecycle

6 requirements marked missing. Six gaps, and the first four are one workstream (task P5). This section contains the only account-takeover path in the audit, and that part is a one-line fix you should make before anything else on this page.

Safe OAuth account linking (do this first)

task P5

WHY IT MATTERS

src/features/authentication/auth.ts sets allowDangerousEmailAccountLinking: true while credential signup never verifies the email address. An attacker registers a password account using someone else's address; when the real owner later signs in with Google, the accounts link, and the attacker's password now opens the victim's account. This is a live takeover path.

STEPS

1. Set allowDangerousEmailAccountLinking: false in the Google provider config. This is a one-line change and closes the hole immediately.
2. Understand the cost: an existing user with both a password account and a Google sign-in for the same address will now see two separate accounts. That is the safe failure, and it is preferable to the current one.
3. Only turn linking back on after email verification is enforced on the credentials side, so both identities are proven.
4. If any accounts were already linked this way, there is no safe automated audit. Treat it as a reason to ship verification promptly rather than to leave linking on.

FILES

`src/features/authentication/auth.ts`

DONE WHEN

The flag is false, sign-in still works for both providers, and a Google sign-in no longer silently adopts an unverified password account.

Email verification

task P5

WHY IT MATTERS

Nothing verifies that a person signing up controls the address they typed. That lets anyone create an account on someone else's address, makes the linking fix above permanent rather than temporary, and means your transactional email goes to unconfirmed addresses, which harms sender reputation.

STEPS

1. Add a verification token store. With no DATABASE_URL this is the file store; with one it is a Postgres table, which means it goes through the migration work in section 5 (task P6). Store a hash of the token, the email, and an expiry of about 24 hours.
2. On successful registration in src/app/api/register/route.ts, generate a single-use token, store its hash, and send the verification email.
3. Add GET /api/auth/verify?token=... that looks up the hash, checks expiry, marks the account verified, and deletes the token. Single use, always.
4. Decide the enforcement point. Blocking all sign-in until verified is strictest; allowing sign-in but gating paid actions is gentler. Pick one and apply it consistently.
5. Add a resend path with its own rate limit so the endpoint cannot be used to send mail at someone.

FILES

`src/app/api/register/route.ts, new verify route, token store, src/services/email/email.ts`

DONE WHEN

A new signup receives a real email, the link verifies exactly once, a reused or expired link is rejected cleanly, and the chosen enforcement applies.

Password reset / forgot password

task P5

WHY IT MATTERS

A grep for forgot-password, reset-password or verification token across src returns nothing. A user who forgets their password is permanently locked out of their own resumes with no self-service recovery. On a paid product this is a support and refund problem from the first week.

STEPS

1. Build it on the same token store as verification. The mechanics are identical: hashed token, short expiry (one hour is appropriate for reset), single use.
2. Add POST /api/auth/forgot that accepts an email and always returns the same success response whether or not the account exists. Revealing which addresses are registered is an enumeration leak.
3. Rate limit that endpoint by email and by the corrected client identity from section 7, so it cannot be used to flood someone's inbox.
4. Add a reset page that accepts the token, validates it server-side before showing the form, and applies the same password policy as registration (reuse registerBodySchema from src/validation/schemas).
5. On success, delete the token and invalidate existing sessions, so a stolen session does not survive a password change.
6. Add a link from /login. A reset flow nobody can find is not a reset flow.

FILES

`new forgot and reset routes and page, token store, src/app/(public)/login`

DONE WHEN

A real reset email arrives, the link works exactly once, the new password signs in, an expired link fails cleanly, and the endpoint does not reveal whether an address is registered.

Change password while signed in

task P5

WHY IT MATTERS

There is no change-password UI or endpoint. Even a user who knows their password cannot rotate it, which matters after a shared device or a suspected compromise.

STEPS

1. Add a section to src/features/user-profile/components/account-settings.tsx, which already handles name editing and deletion.
2. Add PATCH /api/account/password. Require the current password and verify it with bcrypt.compare before accepting the new one. Never allow a change on session alone.
3. Reuse the registration password policy so rules are consistent.
4. Skip or hide the control for OAuth-only accounts, which have no stored password hash.
5. Send the password-changed notification described in section 3, so a user learns if someone else did it.

FILES

`src/features/user-profile/components/account-settings.tsx, src/app/api/account/password/route.ts`

DONE WHEN

A signed-in user can change their password with the current one, a wrong current password is rejected, and OAuth accounts do not see a broken control.

Two-factor authentication

task optional

WHY IT MATTERS

No TOTP or MFA code exists. This is not a launch blocker for a resume product, and it is listed so the decision is explicit rather than accidental.

STEPS

1. Decide and record whether 2FA is in scope for launch. For this product, deferring is defensible.
2. If deferred, ensure the basics that substitute for it are solid: the login brute-force brake already exists, and password reset plus verification close the main recovery risks.
3. Revisit if you ever store payment details directly or add team or admin accounts.

FILES

`none for launch`

DONE WHEN

A recorded decision, not an oversight.

Security event audit log

task optional

WHY IT MATTERS

Nothing records sign-ins, failed attempts, password changes, email changes or deletions. Without it you cannot answer a user asking what happened to their account, and you cannot investigate an incident after the fact.

STEPS

1. Decide the minimum set worth recording: sign-in success and failure, password change, email verification, account deletion, and subscription change.
2. Write them to a dedicated table (migration, per section 5) with timestamp, event type, account identifier and source address. Keep it to facts, never passwords or tokens.
3. Set a retention period and state it in the privacy policy, so the log does not become its own compliance problem.
4. Plain and structured is enough. This does not need to be a product feature.

FILES

`new audit table and a small helper in src/services`

DONE WHEN

The listed events produce rows, the rows contain no secrets, and the retention period is documented.

3. Email and notifications

7 requirements marked missing. Only two emails exist today: welcome and payment receipt. Six more are missing, five of which the product's own behaviour already implies. One is a documentation gap. All of them share one sender and one helper, so this section is largely a single afternoon once the first one is wired.

The shared groundwork

task M5

WHY IT MATTERS

Every item below uses sendEmail in src/services/email/email.ts, which already reads EMAIL_FROM and degrades safely when Resend is unconfigured. Setting the sender up correctly once makes the rest mechanical.

STEPS

1. Add EMAIL_FROM to .env.example and .env.local.example. It is read by the code today but documented in neither template, so a fresh deployment silently falls back to onboarding@resend.dev.
2. Verify your sending domain in Resend and publish SPF, DKIM and DMARC records. Without these, every email below lands in spam and the work is wasted.
3. Follow the existing sendWelcomeEmail shape for each new message: a small function in the same file, plain text alongside HTML, and errors swallowed so a mail failure never breaks the calling flow.
4. Keep the subject lines plain and specific. 'Your payment did not go through' beats 'Important account notice'.

FILES

`.env.example, .env.local.example, src/services/email/email.ts, DNS`

DONE WHEN

EMAIL_FROM is documented, the sending domain passes SPF, DKIM and DMARC checks, and a test send arrives in an inbox rather than spam.

Email verification and password reset messages

task P5

WHY IT MATTERS

These are the delivery half of the two flows in section 2. The flows cannot ship without them.

STEPS

1. Write sendVerificationEmail(to, url) and sendPasswordResetEmail(to, url) next to the existing senders.
2. State the expiry in the body ('this link works for the next hour') so a user who comes back late understands why it failed.
3. Include a plain-text fallback URL, because some clients strip links.
4. For the reset email, add a line telling the user to ignore it if they did not request it. That single sentence prevents a lot of support contact.
5. Never include the raw token anywhere except the link itself, and never log the email body.

FILES

`src/services/email/email.ts`

DONE WHEN

Both messages arrive, render in a plain-text client, and their links work once.

Payment failed notice

task P5

WHY IT MATTERS

src/app/api/stripe/webhook/route.ts already handles invoice.payment_failed and silently sets the subscription to past_due. The user loses access with no explanation and no way to know a card needs updating. This is the highest-value missing email: it is the one that recovers revenue.

STEPS

1. In the invoice.payment_failed branch of the webhook, call a new sendPaymentFailedEmail after the entitlement update.
2. Say what happened, what it means for their access, and exactly how to fix it. Link to the billing portal from section 4.
3. Wrap the send in its own try/catch, exactly as the existing receipt email is wrapped, so a mail failure never fails the webhook and causes Stripe to retry the whole event.
4. Consider Stripe's built-in dunning emails as a complement, not a replacement: yours can link into your own billing screen.

FILES

`src/app/api/stripe/webhook/route.ts, src/services/email/email.ts`

DONE WHEN

Triggering a failed payment in Stripe test mode produces one email with a working link to fix the card, and the webhook still returns 200.

Trial ending reminder

task P5

WHY IT MATTERS

The trial plan converts to a recurring charge after seven days. Charging someone who forgot they signed up is the single largest driver of chargebacks and disputes for trial-based subscriptions, and several jurisdictions now require advance notice.

STEPS

1. Handle the customer.subscription.trial_will_end event, which Stripe sends three days before conversion. Enable it on your webhook endpoint first: it is not on by default.
2. Send an email stating the exact amount, the exact date, and a one-click way to cancel.
3. Do not make it a marketing email. A clear reminder reduces disputes; a persuasive one increases them.
4. This depends on the price being correct, so complete task P4 in section 4 first or the reminder will quote the wrong figure.

FILES

`src/app/api/stripe/webhook/route.ts, src/services/email/email.ts, Stripe dashboard event selection`

DONE WHEN

The trial_will_end event is enabled, triggers one email, and the quoted amount matches what Stripe will charge.

Subscription cancelled notice

task P5

WHY IT MATTERS

customer.subscription.deleted already revokes access in the webhook. The user is told nothing, so a cancellation they did not expect looks like a bug.

STEPS

1. Send from the customer.subscription.deleted branch, after the entitlement update.
2. Confirm what was cancelled, when access ends, and that no further charges will occur.
3. Say explicitly that their documents remain in the account. On a resume product, the first fear is losing the work.
4. Same local try/catch pattern as the other webhook emails.

FILES

`src/app/api/stripe/webhook/route.ts, src/services/email/email.ts`

DONE WHEN

Cancelling a test subscription produces one clear confirmation email.

Account deletion confirmation

task P5

WHY IT MATTERS

Deletion is irreversible and currently silent. A confirmation is both a courtesy and a security signal: it is how a user finds out somebody else deleted their account.

STEPS

1. Send from DELETE /api/account, after the Stripe cancellation succeeds and before or immediately after the data delete.
2. Capture the address before deleting the record, or you will have nothing to send to.
3. Confirm what was removed and that any subscription was cancelled.
4. Complete the deletion work in section 5 (task P3) first, so the email does not claim an erasure that did not happen.

FILES

`src/app/api/account/route.ts, src/services/email/email.ts`

DONE WHEN

Deleting a test account sends one confirmation, and its claims match what the code actually removed.

4. Payments, subscription and entitlement

10 requirements marked missing. The largest section, and the one carrying the most commercial risk. Two items are launch blockers on their own: the paywall is not enforced anywhere a user cannot edit, and the price shown at checkout is not the price Stripe is asked to charge. Work this section in the order given; several items share files.

Displayed price equals charged price (do this first)

task P4

WHY IT MATTERS

src/services/payments/plans.ts advertises the trial as priceLabel '$1.95' and summaryAmount '$1.95'. The Stripe payload for the same plan is unitAmount 2995 with trial_period_days 7. The customer is charged $0.00 today and $29.95 in seven days. The figure $1.95 is never sent to Stripe at all. A displayed price that does not match the charge invites chargebacks, breaches consumer-disclosure rules, and is a common trigger for Stripe account review.

STEPS

1. Decide with the business which product you are actually selling: a genuinely paid $1.95 trial, or a free seven-day trial that converts to $29.95.
2. For a paid trial, add a one-off $1.95 line item and remove trial_period_days, so money actually moves on day one.
3. For a free trial, change every label to state '$0.00 today, then $29.95 per month after 7 days'.
4. Derive every displayed figure from the same plan object the Stripe call uses. The bug exists because the labels are hand-written strings sitting beside the real amounts. Make the wrong state unrepresentable.
5. Check the annual plan while you are there: confirm the $119.40 label matches unitAmount 11940.
6. Do this before the public pricing page in section 1 and the trial reminder in section 3, both of which quote these figures.

FILES

`src/services/payments/plans.ts, payment and checkout pages`

DONE WHEN

A Stripe test-mode checkout charges exactly what the summary said, for both plans, and no price string in the codebase is written independently of the Stripe amount.

Entitlement enforced on paid features

task P1, P2

WHY IT MATTERS

Premium is a flag in localStorage under resume-co:paywall. Editing it in devtools, or calling usePaywall.setState({premium:true}) in the console, unlocks everything. Worse, all seven download entry points funnel into exportElementToPdf, which renders the PDF in the browser, so there is no server step to gate. The monetisation model is currently unenforceable, and this is a product decision before it is a code change.

STEPS

1. Decide the model (task P1). Either move rendering server-side behind an entitlement check, or redefine the free tier as what you accept giving away and monetise something that genuinely requires your server, such as AI tailoring or job matching.
2. If you choose server rendering: add a route that authenticates, calls isPremium(session.user.email) from src/permissions/entitlements.ts, renders the PDF server-side and streams it. Returning 401 or 403 otherwise.
3. Expect real work here: moving off html2canvas-pro in the browser to a server renderer must reproduce the current visual output, which is the hard part.
4. Demote the client store to presentation only. Keep it for showing the upgrade dialog; never let it decide whether an action is allowed.
5. Audit the five consumers so none gates a real capability on the local flag: unlock-premium.tsx, paywall-dialog.tsx, dashboard-resumes.tsx, resume-card.tsx, documents-store.ts.
6. Copy the pattern already correct in PUT /api/documents, which re-checks entitlement server-side rather than trusting the client.

FILES

`src/features/cover-letter/lib/paywall.ts, src/services/storage/*, 5 consumer components, new route`

DONE WHEN

With localStorage hand-edited to premium, the gated action still fails. Revoking the subscription in Stripe removes access without the user clearing storage.

Session required at checkout, and entitlement keyed properly

task P7

WHY IT MATTERS

In src/app/api/checkout/session/route.ts, auth() is read but never required, so client_reference_id and the subscription metadata are omitted for anonymous buyers. The webhook then falls back to customer_details.email, the address the payer typed into Stripe's form. A mistyped address means the payer pays and receives nothing with no link to any account; someone else's address grants premium to that account instead.

STEPS

1. Return 401 from the checkout route when there is no session. Checkout should require an account.
2. In the webhook, accept only client_reference_id and subscription metadata that your own server set. Remove the customer_details.email fallback.
3. Longer term, key the subscriptions table on a stable user id rather than a mutable email. That is a migration plus a backfill, so it rides on section 5.
4. Add a rate limit to the checkout route. It has none today, so session creation can be spammed.
5. Do this before or with the webhook work below: both touch the same two files, so ship them as one change.

FILES

`src/app/api/checkout/session/route.ts, src/app/api/stripe/webhook/route.ts`

DONE WHEN

An anonymous POST to the checkout route is refused, and entitlement cannot be granted to an address the server did not supply.

Webhook idempotency and event ordering

task P8

WHY IT MATTERS

Signature verification is already correct: raw body before parsing, async verify, Node runtime pinned, 500-on-error so Stripe retries. Two gaps remain. No event.id is recorded, so Stripe's at-least-once delivery can send a duplicate receipt email. And entitlement writes stamp Date.now() rather than the event timestamp, so a subscription.updated arriving after a subscription.deleted can silently restore cancelled access.

STEPS

1. Add a processed_events table as a numbered migration (section 5, task P6). Insert event.id at the start of handling and return early on conflict.
2. Replace Date.now() with event.created as the write timestamp.
3. Refuse any entitlement write whose event timestamp is older than the stored updated_at, so out-of-order delivery cannot resurrect access.
4. Test with the Stripe CLI rather than by reasoning: stripe listen --forward-to localhost:3001/api/stripe/webhook, then stripe trigger customer.subscription.deleted, then replay an earlier updated event and confirm access stays revoked.

FILES

`src/app/api/stripe/webhook/route.ts, new migration`

DONE WHEN

Replaying the same event twice produces one receipt email and one write. An out-of-order update after cancellation does not restore access.

Self-serve cancellation and billing history

task R2

WHY IT MATTERS

There is no Stripe billing portal session, no cancellation control and no invoice screen anywhere in the app. A subscriber cannot cancel without contacting you. Beyond the support burden, click-to-cancel rules in the EU, UK and California require cancellation to be as easy as signing up. This must land before you take real money.

STEPS

1. Add POST /api/billing/portal that calls stripe.billingPortal.sessions.create for the signed-in user's customer id and returns the URL. The portal gives you cancellation, card updates and invoice history in one step.
2. Enable and configure the customer portal in the Stripe dashboard: allowed actions, cancellation behaviour, and your branding.
3. Add a Billing section to the account page linking to it.
4. Decide whether cancellation is immediate or at period end, and make sure the wording matches the refund policy from section 1.
5. Link to the portal from the payment-failed email in section 3.

FILES

`src/app/api/billing/portal/route.ts, account settings component, Stripe dashboard`

DONE WHEN

A subscriber can cancel and view invoices without contacting support, and the webhook correctly revokes access when they do.

Tax and VAT handling

task open decision

WHY IT MATTERS

The plans carry only a 'Tax may apply' note. If you sell to consumers in the EU, UK or many US states, tax is your obligation to collect and remit, and 'may apply' is not a position.

STEPS

1. Decide where you are selling. This is a business and accounting decision, not an engineering one.
2. If the answer is anywhere beyond your home jurisdiction, enable Stripe Tax and set automatic_tax on the checkout session. It handles calculation and reporting.
3. Set the product tax code so digital services are treated correctly.
4. Update the displayed prices to state tax-inclusive or tax-exclusive clearly, consistent with the fix in the first item of this section.
5. Record the decision even if it is to sell in one country only.

FILES

`src/app/api/checkout/session/route.ts, Stripe dashboard`

DONE WHEN

Tax treatment is a decision with a date on it, and if collected, it appears correctly on a test invoice.

Documented refund process

task M8

WHY IT MATTERS

Covered as a page in section 1. Listed here because the operational half belongs to whoever handles billing: a published policy with no internal process produces inconsistent decisions.

STEPS

1. Agree who can issue a refund and within what window.
2. Write the two-line internal procedure: where to action it in Stripe, and what to record.
3. Make sure a refund revokes entitlement. Check whether your webhook handles charge.refunded; today it does not.
4. Keep the public wording and the internal process in agreement.

FILES

`Stripe dashboard, internal runbook, possibly the webhook`

DONE WHEN

A refund can be issued consistently, and issuing one removes premium access.

5. Data, database and storage

9 requirements marked missing. Nine gaps covering how data is changed, kept and destroyed. Two are serious: account deletion does not delete, and two browser tabs editing one document silently overwrite each other. The migration work comes first because several other sections need a safe way to add a table.

Versioned migrations, a ledger, and a rollback path

task P6

WHY IT MATTERS

database/migrations, policies, functions and seed are all empty. The entire schema is one 105-line schema.sql applied by hand, and it already contains in-place drift repairs (drop constraint if exists) for databases created by earlier runs. There is no way to know what a production database contains, no rollback, and no CI check that code matches schema. Several later tasks need a new table, so this unblocks them.

STEPS

1. Freeze today's schema as database/migrations/0001_init.sql, copied verbatim from schema.sql so a fresh database reaches the current state.
2. Add a schema_migrations ledger table recording filename and applied timestamp.
3. Make the runner idempotent: read the ledger, apply only unapplied files in filename order, record each one. Running it twice must be a no-op the second time.
4. Apply it to a fresh staging database and diff the result against schema.sql. They must match exactly. Any difference is drift you did not know about.
5. Add a CI job that applies migrations to an empty database, so a schema change landing without a migration fails the build rather than production.
6. Decide the rollback convention now. Forward-only with a compensating migration is simpler and safer than down-scripts; whichever you choose, write it down.
7. From here on, every schema change ships as a numbered migration. Do not edit schema.sql by hand again.

FILES

`database/migrations/, scripts/db-setup.mjs, .github/workflows/ci.yml`

DONE WHEN

A fresh database reaches the current schema from migrations alone, running the runner twice is a no-op, and CI fails if a schema change has no migration.

Account deletion must remove all user data

task P3

WHY IT MATTERS

pgDeleteUser runs one statement, delete from users, and its comment claims documents are removed by ON DELETE CASCADE. That cascade does not exist: schema.sql explicitly drops the foreign key so OAuth accounts can own documents. Surviving deletion indefinitely: documents (full resumes with legal name, address, phone, employment history), saved_jobs, dismissed_jobs, tailoring_sessions and subscriptions. Worse, a Google-only account has no users row at all, so for those users the delete is a complete no-op while the UI reports success. The published privacy policy says the opposite.

STEPS

1. Rewrite pgDeleteUser to clear all six tables in one transaction keyed on the normalised email, never conditional on a users row existing. postgres 3.4.9 provides sql.begin; this was verified available.
2. Delete in this order inside the transaction: documents, saved_jobs, dismissed_jobs, tailoring_sessions, subscriptions, then users.
3. Fix fileDeleteUser too. It has the identical hole: it rewrites users.json and never touches documents.json.
4. Decide what happens to the subscriptions row. Cancellation already happens first in DELETE /api/account. Keep a billing record only if you have a stated legal basis, and if you do, say so in the privacy policy.
5. Correct the misleading comment. That comment is what hid this bug.
6. Add a security test that seeds rows in all six tables, deletes the account, and asserts zero rows remain. Run it for an OAuth-style account with no users row, and against the file store backend.

FILES

`src/services/database/users.ts (lines 99 and 185), tests/security/`

DONE WHEN

The new test passes for a credentials account, an OAuth account and the file store. The privacy policy matches what the code now does.

Concurrent-edit protection

task P14

WHY IT MATTERS

Document saves are an unconditional whole-blob overwrite: pgUpsertDocument ends with on conflict (email, type, id) do update set data = excluded.data and no updated_at guard. Open one resume in two tabs, change the title in one and the summary in the other, and whichever saves last replaces the entire document, discarding the other edit with no warning. A slow or retried request can also overwrite a newer save with stale data. With autosave running, neither tab needs an explicit save for this to happen. This was found by the existing QA handbook, not by the audit.

STEPS

1. Add where excluded.updated_at > documents.updated_at to the conflict clause, so a stale write cannot land.
2. Return a distinct status when a write is refused, rather than reporting success.
3. Have the client tell the user their copy is behind and offer to reload, instead of failing silently.
4. Apply the same guard to the file-store backend, which has identical whole-blob behaviour.
5. Decide the product answer for a genuine two-tab conflict: refuse and reload, or merge at section level. Refusing is acceptable; silently discarding is not.
6. Verify with the QA handbook's own test in section 14.1: two tabs, different fields, save both, reload both, and confirm neither edit disappeared.

FILES

`src/services/database/documents.ts lines 198-202, src/app/api/documents/route.ts`

DONE WHEN

Two tabs editing different fields no longer lose either edit, a save with an older updated_at is rejected, and the user is told when a save was refused.

Document blob size cap

task P11

WHY IT MATTERS

PUT /api/documents stores an arbitrary record.data value as jsonb with no size limit, and there is no document-count cap outside resumes. One account can fill the database. Authorization itself is sound: every handler scopes by session email and four isolation tests cover it.

STEPS

1. Validate the body with Zod, as /api/ai already does, including a maximum serialised size. A few hundred kilobytes is generous for a resume.
2. Return 413 with a plain-language message when the limit is exceeded, matching the upload guards already in the AI route.
3. Add a per-account cap for cover letters, resignation letters and interview-prep sheets, not only resumes.
4. Add tests for both caps.

FILES

`src/app/api/documents/route.ts`

DONE WHEN

An oversized document is rejected with 413, per-type caps are enforced, and both have tests.

Seed data

task optional

WHY IT MATTERS

database/seed is empty. Not a launch blocker, but it makes local development and the end-to-end tests in section 10 harder than they need to be, because every run starts from nothing.

STEPS

1. Add a seed script creating one known test account with a sample resume, a saved job and a tailoring session.
2. Keep it obviously fictional. Never seed real personal data.
3. Make it idempotent so it can run before each test suite.
4. Guard it so it refuses to run when NODE_ENV is production.

FILES

`database/seed/`

DONE WHEN

A fresh developer can reach a populated app in one command, and the seed cannot run against production.

Backups and a rehearsed restore

task M9

WHY IT MATTERS

No backup or restore plan is documented and no restore has ever been performed. An untested backup is not a backup. This is the item most often discovered to be broken at the worst possible moment.

STEPS

1. Confirm what your database provider actually retains by default, and for how long. Do not assume.
2. Write down the recovery point and recovery time you are willing to accept, then check the provider's defaults meet them.
3. Perform a real restore into a scratch database. This is the step that counts; everything before it is paperwork.
4. Time it and write the procedure down as a runbook, including who runs it and what they need access to.
5. Schedule a repeat. A restore rehearsed once a year is the minimum that keeps the runbook honest.

FILES

`database provider settings, operations runbook`

DONE WHEN

A restore has actually been performed, timed and documented, not merely configured.

Virus scanning on upload

task optional

WHY IT MATTERS

Uploaded files go straight to the AI model. Type and size are already checked server-side, which blocks the obvious cases, but no content scanning happens. Risk is moderate: files are not stored or served back to other users, which is what usually makes this dangerous.

STEPS

1. Record the risk decision explicitly rather than leaving it unexamined.
2. If you add scanning, do it after the existing type and size checks in the AI route and before the model call.
3. Revisit this immediately if you ever store uploads or serve them back to users, which changes the risk profile completely.

FILES

`src/app/api/ai/route.ts`

DONE WHEN

A recorded decision with a stated trigger for revisiting it.

6. Privacy and compliance

7 requirements marked missing. Seven gaps. Two of them are promises the product already makes and does not keep, which is the most dangerous kind: the privacy policy states that content is kept until a user deletes it, and deletion does not delete. Fix the code first, then the words.

Make the policy match the implementation, and honour erasure

task P3

WHY IT MATTERS

The published policy says 'We keep your account and content until you delete them.' Deletion removes only the users row, so that sentence is currently untrue. These two inventory rows are one problem with one fix, and the fix is the deletion work in section 5.

STEPS

1. Complete the deletion fix in section 5 first. Rewriting the policy to match broken behaviour would be the wrong repair.
2. Once deletion is complete, re-read the policy line by line against what the code does, not what you intended it to do.
3. State what survives deletion and why, if anything does. A retained billing record with a stated legal basis is defensible; an unexplained one is not.
4. Add the deletion confirmation email from section 3, so the user has a record.
5. Keep a dated note of when the policy was last reconciled with the code.

FILES

`src/app/(public)/privacy/page.tsx, src/services/database/users.ts`

DONE WHEN

Every claim in the policy is true of the code as shipped, verified by reading them side by side.

Cookie consent before analytics

task M3

WHY IT MATTERS

PostHog and Sentry initialise unconditionally in src/instrumentation-client.ts. For any EU or UK visitor, non-essential storage set before consent is an ePrivacy breach from the first page view.

STEPS

1. Classify each storage honestly: the Auth.js session cookie is strictly necessary and needs no consent. PostHog and Sentry session replay are not.
2. Add a consent banner offering a genuine choice. Reject must be as easy as accept; a banner with only an accept button does not constitute consent.
3. Gate the analytics initialisation behind the stored choice. Do not load PostHog and then opt out afterwards, which still sets storage.
4. Persist the choice and provide a way to change it later, linked from the cookie policy page in section 1.
5. Verify in devtools that no analytics request fires before a choice is made. That is the only test that matters.

FILES

`src/instrumentation-client.ts, new consent component, src/components/providers.tsx`

DONE WHEN

With a fresh browser profile, no PostHog or Sentry request appears in the network tab until consent is given, and rejection is one click.

Complete processor disclosure, retention and legal basis

task M4

WHY IT MATTERS

The 147-line policy names Stripe, Gemini, Google and Adzuna but omits Resend, PostHog, Sentry, Dropbox and Upstash. It states no retention period and no legal basis. For a product holding resumes, which routinely contain data that is sensitive in practice, this is thin.

STEPS

1. List every third party that receives user data. From the source that is: Stripe, Google Gemini, Google OAuth, Adzuna, Resend, PostHog, Sentry, Dropbox and Upstash.
2. For each, state what is shared and why. Gemini matters most to be clear about: users are sending it their employment history.
3. State retention: how long documents, accounts and logs are kept, and what happens after deletion.
4. State the legal basis for each processing purpose. For a paid product this is usually contract for the service and consent for analytics.
5. Name the data controller and say where it is established.
6. Have someone qualified review it. This is the one item on this page where engineering judgement is not sufficient.

FILES

`src/app/(public)/privacy/page.tsx`

DONE WHEN

Every processor the code actually calls is named, retention and legal basis are stated, and the controller is identified.

Privacy contact route

task M4

WHY IT MATTERS

There is no address or form for a privacy request. A user exercising a legal right has nowhere to send it, and the obligation to respond runs from the moment they try.

STEPS

1. Use the same address as the contact page in section 1. One monitored inbox satisfies both.
2. Publish it in the privacy policy explicitly as the route for data requests.
3. Agree internally who monitors it and what the response deadline is. Under GDPR it is one month.
4. Write down the procedure for handling an access or deletion request, including how you verify the requester is who they claim to be.

FILES

`src/app/(public)/privacy/page.tsx, internal runbook`

DONE WHEN

A named inbox exists, is monitored, and a documented procedure covers access, deletion and identity verification.

Data export and portability

task R3

WHY IT MATTERS

There is no export endpoint. Portability is a legal right where GDPR applies, and the technical work is small because the data model is already keyed by email.

STEPS

1. Add GET /api/account/export returning everything held for the signed-in account as JSON: documents, saved and dismissed jobs, tailoring sessions, subscription status and profile.
2. Use the same six tables as the deletion fix. If you keep the two lists in one place, they cannot drift apart.
3. Rate limit it; it is an expensive query.
4. Add a download control to the account page beside the delete control.
5. A machine-readable JSON file satisfies the requirement. It does not need to be pretty.

FILES

`src/app/api/account/export/route.ts, account settings component`

DONE WHEN

A signed-in user can download a complete JSON file of their data, and its contents match what deletion removes.

7. Security

5 requirements marked missing. Five gaps. The headers, secret hygiene, parameterised SQL and tenancy scoping are already in good shape, so this section is short. One item costs real money if left: the rate-limit identity is supplied by the caller.

Trustworthy rate-limit identity

task P9

WHY IT MATTERS

clientIp() in src/permissions/rate-limit.ts line 155 returns the first entry of X-Forwarded-For, which the caller controls. Rotating that header per request defeats every IP-keyed limit. The AI endpoint accepts anonymous requests and is keyed on exactly this value, so your Gemini spend is uncapped by anyone willing to set a header. The same gap affects /api/jobs (Adzuna quota) and /api/register (signup spam).

STEPS

1. Trust only the platform-provided client address. On Vercel that is the last hop or the platform header, never the first entry a client can prepend.
2. Require a session for /api/ai, or give anonymous callers a much smaller budget keyed separately.
3. Treat Upstash as required in production. Without UPSTASH_REDIS_REST_URL and _TOKEN every limiter falls back to a per-instance Map, which on serverless makes both the login brake and the AI quota close to meaningless.
4. GET /api/health already reports sharedRateLimit. Alert on it being false rather than discovering it from a bill.
5. Verify by rotating the header against a local endpoint and confirming the quota still applies.

FILES

`src/permissions/rate-limit.ts, src/app/api/{ai,jobs,register}/route.ts`

DONE WHEN

Header rotation no longer raises the effective quota, and production refuses to run, or alerts loudly, with in-memory rate limiting.

Remove the unauthenticated scraping proxy

task P10

WHY IT MATTERS

POST /api/linkedin fetches a LinkedIn profile page server-side and returns its text, with no session and no rate limit. Anyone on the internet can use your deployment as a scraping proxy, from your IP and under your platform account. Server-side scraping of profile pages also conflicts with LinkedIn's terms. The URL regex is anchored to linkedin.com/in/, so SSRF is contained; the abuse and legal exposure are not.

STEPS

1. Delete the route from the launch build. Project notes already record this feature as held pending a real LinkedIn API key, so the endpoint should be held with it.
2. If it must stay for internal testing, require a session and a rate limit, and exclude it from production builds.
3. This is a pure deletion, so it conflicts with nothing. Do it early.

FILES

`src/app/api/linkedin/route.ts`

DONE WHEN

The route returns 404 in a production build.

script-src Content Security Policy

task M6

WHY IT MATTERS

next.config.ts sets nosniff, DENY framing, strict-origin referrer, a Permissions-Policy, two-year HSTS and frame-ancestors none, but no script-src. The file documents why: a real script-src needs per-request nonces via middleware and would otherwise break Next's inline hydration and libraries like framer-motion and tiptap. This is a reasoned trade-off, not an oversight, and is listed so it stays tracked.

STEPS

1. Implement nonce generation in src/proxy.ts, Next 16's renamed middleware, and pass the nonce through to the script tags.
2. Roll out with Content-Security-Policy-Report-Only first and collect violations for a week. Deploying a blocking CSP straight to production reliably breaks something.
3. Fix the reported violations, then switch to enforcing.
4. Keep the existing headers exactly as they are. They are correct.
5. If this slips past launch, that is defensible given the other protections; leaving it undocumented is not.

FILES

`next.config.ts, src/proxy.ts`

DONE WHEN

A report-only policy runs with no unexplained violations, then enforces without breaking hydration, motion or the editor.

Credential rotation procedure

task optional

WHY IT MATTERS

No documented procedure exists for rotating the keys the product depends on: AUTH_SECRET, Stripe keys, Gemini, Resend, Upstash and the database URL. You will need this the first time a laptop is lost or someone leaves.

STEPS

1. List every secret, where it lives and who can change it.
2. Write the rotation order for each, noting side effects. Rotating AUTH_SECRET invalidates every session, so users are signed out; that is acceptable but should not be a surprise.
3. Note which can be rotated with zero downtime and which need a deploy.
4. Keep it in the operations runbook next to the rollback procedure from section 11.

FILES

`operations runbook`

DONE WHEN

Any secret can be rotated by following written steps, with the user-visible effect stated in advance.

Penetration test

task optional

WHY IT MATTERS

No security testing beyond this source review has been performed. For a product holding resumes and taking payments, an external pass before or shortly after launch is proportionate.

STEPS

1. Close the items above first. Paying someone to find what you already know is waste.
2. Scope it to the authenticated application, the payment flow and the AI endpoints.
3. The negative tests in section 10 cover the cheap cases and should be written first.
4. Budget for remediation time, not just the test.

FILES

none

DONE WHEN

A scoped test has been commissioned, or a dated decision to defer exists.

8. SEO, discovery and advertising

5 requirements marked missing. Technical SEO is already correct: metadata, a title template, robots.txt disallowing every per-account surface, and a sitemap listing the public routes. These five gaps are about how the product appears when shared, and whether it can be advertised at all.

Open Graph share image

task R4

WHY IT MATTERS

The layout sets Open Graph and Twitter titles and descriptions but no image. Every link shared to LinkedIn, Slack or WhatsApp renders as a bare text card. For a product whose users are actively sharing career material, that is a visible quality signal.

STEPS

1. Add src/app/opengraph-image.tsx using Next's built-in image generation, or a static 1200x630 PNG in the same place.
2. Use the design tokens already extracted in design-system/: the brand blue and Gabarito heading face, so the card matches the product.
3. Add a matching twitter-image, or let Next fall back to the Open Graph one.
4. Give per-page images only where they add something. One good default beats five mediocre variants.
5. Test with a real link preview debugger, not by eye.

FILES

`src/app/opengraph-image.tsx, src/app/layout.tsx`

DONE WHEN

Pasting the site URL into LinkedIn and Slack renders a correct branded card.

JSON-LD structured data

task R4

WHY IT MATTERS

No schema.org markup exists anywhere. Structured data is how search engines understand what a page is, and for a SaaS product with pricing it affects how results are displayed.

STEPS

1. Add Organization markup to the root layout: name, URL, logo and contact.
2. Add SoftwareApplication or Product markup to the pricing page from section 1, including the offer, once the prices are correct.
3. Add FAQPage markup only if you actually publish FAQs. Marking up content that is not on the page is a policy violation.
4. Inject it as a script tag with type application/ld+json.
5. Validate with Google's Rich Results Test before shipping.

FILES

`src/app/layout.tsx, pricing page`

DONE WHEN

The Rich Results Test passes with no errors, and the marked-up content is genuinely present on each page.

Content depth for advertising review

task M8

WHY IT MATTERS

Four public pages with no articles will not pass AdSense review, and thin public content also limits organic acquisition. The pages added in section 1 help but will not be sufficient on their own.

STEPS

1. Recognise this is a content programme, not an engineering task. No code change makes a four-page site reviewable.
2. If advertising revenue is a goal, plan genuinely useful published content: resume guidance, interview preparation, job-search material. The product already generates expertise in these areas.
3. If advertising is not a goal, record that and drop both this item and ads.txt from the list rather than leaving them open.
4. Publishing the About, Contact, Pricing and Refund pages from section 1 first is a prerequisite either way.

FILES

`content, not code`

DONE WHEN

Either a content plan exists with dates, or a recorded decision not to pursue ad revenue.

Web app manifest

task optional

WHY IT MATTERS

No manifest.json exists, so the product cannot be installed to a home screen and has no defined icon or theme colour for mobile browsers. Minor, and cheap.

STEPS

1. Add src/app/manifest.ts using Next's metadata route.
2. Set name, short name, start URL, display, theme colour and background colour. Use the brand blue from the token layer.
3. Supply the standard icon sizes.
4. Skip the service worker. Offline support is a real feature with real cost and is not implied by a manifest.

FILES

`src/app/manifest.ts, public icons`

DONE WHEN

The manifest validates and mobile browsers pick up the correct name, icon and theme colour.

ads.txt

task conditional

WHY IT MATTERS

Only needed if you monetise with ads. Listed so the dependency is explicit.

STEPS

1. Skip entirely unless the content decision above says you are pursuing ad revenue.
2. If you are, publish ads.txt at the domain root with your publisher id once approved.

FILES

`public/ads.txt`

DONE WHEN

Present and correct if advertising, deliberately absent otherwise.

9. UX, mobile and accessibility

2 requirements marked missing. Only two rows are marked missing, but three more are blocked for want of testing. The code already does several things right: focus is never removed without a replacement ring, all motion sits behind prefers-reduced-motion, and scrollbar-gutter is stable so pages do not shift sideways. What is absent is any evidence that the result works.

Skip-to-content link

task R5

WHY IT MATTERS

There is no skip link in src/app/layout.tsx. Keyboard and screen-reader users must traverse the entire sidebar navigation on every page before reaching content. This is the single cheapest accessibility fix available and it is conventionally the first one checked.

STEPS

1. Add an anchor to #main as the first focusable element in the body, before the shell renders.
2. Style it visually hidden until focused, then clearly visible. The existing sr-only pattern plus a focus-visible override is enough.
3. Give the main landmark a matching id and ensure it is a real <main> element.
4. Test by loading a page and pressing Tab once. The link must appear and must work.

FILES

`src/app/layout.tsx, app shell component`

DONE WHEN

Tab from a fresh page load reveals a working skip link that moves focus past the navigation.

Accessibility audit and the blocked UX checks

task R5

WHY IT MATTERS

No audit tool has ever been run against this UI. Three further inventory rows are blocked rather than missing because they cannot be judged from source: responsive behaviour, keyboard navigation and focus, and alt-text coverage. Both QA handbooks also require dialogs to trap focus, close on Escape and restore focus on close, which has never been verified.

STEPS

1. Run axe or Lighthouse against each main surface: landing, login, dashboard, the resume editor, jobs, checkout and account.
2. Fix the one outstanding jsx-a11y/role-supports-aria-props lint error while you are in this area; it is part of the 62 real errors in section 11.
3. Check every dialog and popover for focus trapping, Escape to close, and focus returning to the trigger. Base UI gives you much of this, but it must be confirmed.
4. Test at the widths the usability handbook specifies: 280, 320 and 414 pixels, plus tablet and desktop. The sidebar shell at mobile width is the highest-value unknown, since every authenticated page sits inside it.
5. Audit alt text on the illustrations and template thumbnails. Decorative images should be marked as such rather than given invented descriptions.
6. Record one known deliberate exception: white on the bright brand blue in filled buttons measures 3.31 to 1, below AA. The source documents it as an accepted trade-off scoped to the button fill. Do not silently change it, and do not report it as a new finding.

FILES

`across the UI; see design-system/DESIGN.md section 17`

DONE WHEN

An audit has been run and its findings triaged, dialogs behave correctly, and the three blocked rows have real answers rather than assumptions.

10. Testing

5 requirements marked missing. 113 unit and isolation tests pass today, and the four tenancy-isolation tests are genuinely good. What is missing is any evidence that a user can complete a task. Playwright is installed and entirely unused. This section is what keeps every other fix on this list from quietly regressing.

End-to-end journey tests

task P13

WHY IT MATTERS

tests/end-to-end is empty. Not one of signup, login, build a resume, download, checkout, or delete account has ever been verified as a flow. This is the last task in the execution order deliberately: a spec written before its fix tests nothing, and these specs are what hold sections 2 to 6 in place.

STEPS

1. Write six specs: signup, login, build and download a resume, checkout, entitlement revocation, and account deletion.
2. Authenticate correctly or the specs will not work: set the cookie authjs.session-token AND stub page.route for the session endpoint. NextAuth discards a forged cookie on reload, so the cookie alone is not enough. This is recorded in the project notes and has cost time before.
3. Make the paywall and deletion specs adversarial, because they are the regression guards for the two biggest fixes. In the download spec, hand-edit localStorage to premium and assert the gated action still fails. In the deletion spec, assert that every table is empty afterwards.
4. Add the two-tab conflict case from section 5, which is exactly the kind of bug a unit test will never find.
5. Wire them into CI against the dev server.
6. Verify the suite has teeth: revert one Wave 1 fix on a scratch branch and confirm a spec turns red. A green suite that cannot fail is worse than none.

FILES

`tests/end-to-end/, .github/workflows/ci.yml`

DONE WHEN

Six specs green in CI, and reverting any major fix turns one of them red.

Integration tests

task P13

WHY IT MATTERS

tests/integration is empty. The gap between unit tests and full browser journeys is where API-level behaviour lives: status codes, validation, authorization and error shapes.

STEPS

1. Cover each API route's contract: documents, account, entitlement, checkout, jobs and the AI endpoint.
2. Assert the status codes the QA handbook documents: 400 validation, 401 unauthenticated, 403 including resume_limit, 413 and 415 upload guards, 429 rate limited, 503 when Stripe is unconfigured.
3. Test against the file-store backend so the suite needs no database, keeping it fast enough to run on every commit.
4. Include the webhook signature path: a request with a bad signature must be rejected.

FILES

`tests/integration/`

DONE WHEN

Every API route has its contract asserted, and the documented status codes are the ones actually returned.

Negative and abuse tests

task P13

WHY IT MATTERS

No tampering or authorization-bypass cases exist. The isolation tests prove the happy path is scoped correctly; nothing proves the system refuses an attacker.

STEPS

1. Add the QA handbook's user-to-user test as an automated case: user A creates a resume, user B requests it, and B must receive 403 or 404 with nothing leaking before the error.
2. Add a paywall-tampering case: a forged localStorage premium flag must not unlock a gated action.
3. Add a rate-limit case: rotating X-Forwarded-For must not raise the effective quota, which is the regression guard for section 7.
4. Add an oversized-document case asserting 413.
5. Add a stale-write case asserting the updated_at guard from section 5 rejects an old save.

FILES

`tests/security/`

DONE WHEN

Each of the five refusals is asserted by a test, and each test fails if its protection is removed.

Load and performance testing

task optional

WHY IT MATTERS

Never performed. No measurement of behaviour under concurrency, and no idea where the AI endpoint or PDF generation falls over.

STEPS

1. Defer until after launch unless you expect a large launch-day spike.
2. When you do it, target the expensive paths: the AI endpoint and whatever the paywall decision in section 4 does to PDF generation.
3. Decide what good looks like before measuring, or the numbers will not mean anything.

FILES

`none for launch`

DONE WHEN

A dated decision to defer, or results with a stated target.

Coverage reporting

task optional

WHY IT MATTERS

No coverage gate exists. Useful as information, harmful as a target: a percentage threshold reliably produces tests written to satisfy the number.

STEPS

1. Enable Vitest coverage reporting so the figure is visible in CI output.
2. Do not set a failing threshold initially. Watch the trend instead.
3. Pay attention to which files have no coverage at all, which is the signal that matters more than the percentage.

FILES

`vitest config, CI workflow`

DONE WHEN

Coverage is reported and visible, without being enforced as a target.

11. Build, CI/CD and environments

4 requirements marked missing. Four gaps, and together they mean there is currently no way to deploy this product safely. There is no pipeline, no staging environment, no environment separation and no rollback procedure. Also covered here is the lint gate, which is red today and is the single biggest source of merge conflicts if left until later.

Get the lint gate green (sequencing matters)

task P12

WHY IT MATTERS

npm run lint exits non-zero with 189 errors and 1,525 warnings, and the CI quality job runs lint as a required step, so CI fails on this branch today and protects nothing. Most of the volume is not application code: ESLint is reading ds-bundle, design-ref, .ds-sync and agent-skills, including a minified bundle. Linting src alone gives 62 errors, and those are real: one rules-of-hooks violation and 38 set-state-in-effect.

STEPS

1. Add the design-reference directories to globalIgnores in eslint.config.mjs. One file, two minutes, and it drops 189 errors to 62 real ones so CI starts telling the truth.
2. Fix the rules-of-hooks violation first. That one is a genuine runtime hazard, not a style complaint.
3. Work through the remaining errors in src. They span up to 85 files.
4. Do this before any feature work begins. Mechanical edits across 85 files will conflict with every open branch, so running it mid-stream guarantees painful merges.
5. Once the tree is clean, remove continue-on-error from the format check so formatting is enforced too.
6. Resolve the Node version drift: CI pins 20, developers are on 22, the hosting runtime is unset. Pin all three to the same major.

FILES

`eslint.config.mjs, up to 85 files in src, .github/workflows/ci.yml`

DONE WHEN

All four gates pass on a pull request into main, and branch protection requires them before merge.

Deploy pipeline

task R6

WHY IT MATTERS

There is no vercel.json, no Dockerfile and no deploy job in CI. Deployment today would be a manual action nobody has written down, which is also why no rollback exists.

STEPS

1. Choose the target and commit to it. The codebase assumes Vercel in places, which is the path of least resistance.
2. Connect the repository so pull requests produce preview deployments. Previews are what make the staging work below cheap.
3. Set the production branch to main and require the CI gates before a production deploy.
4. Pin the Node runtime version to match CI.
5. Deploy from a tag rather than a branch tip, so what is in production is always identifiable.
6. Confirm the build command and environment variables, remembering AUTH_SECRET must be present at build time.

FILES

`hosting configuration, .github/workflows/`

DONE WHEN

A tagged commit deploys through the pipeline with no manual steps, and the deployed version is identifiable from the tag.

Staging environment and environment isolation

task R6

WHY IT MATTERS

No staging environment has ever been stood up, and no separation between production and staging credentials is defined. Without staging you cannot rehearse a migration, verify a journey, or test a webhook before real customers meet it. Without separation, a staging mistake reaches production data.

STEPS

1. Create a staging deployment with its own database, separate from production. The migration work in section 5 is what makes this repeatable.
2. Give staging its own Stripe test-mode keys and its own webhook endpoint. Never point staging at live Stripe.
3. Give it a separate Upstash instance and a separate Resend sending configuration, so a staging email run cannot damage your production sender reputation.
4. Set NEXT_PUBLIC_SITE_URL correctly per environment; it feeds canonical URLs and the sitemap.
5. Add robots noindex on staging. A staging environment indexed by Google is a real and common problem.
6. Document which secrets exist in which environment, and make sure no production secret is reachable from staging.

FILES

`hosting environment configuration, DNS`

DONE WHEN

Staging runs the full product against its own data and test keys, nothing in staging can touch production, and staging is not indexable.

Rollback runbook

task R6

WHY IT MATTERS

No rollback procedure is documented. The time to work out how to undo a deploy is not during a bad deploy.

STEPS

1. Write down how to revert a deployment on your chosen platform, with the exact steps and who can perform them.
2. Cover the database separately. Code rollback is easy; a migration that has already run is not. State which migrations are reversible and which need a compensating forward migration.
3. Define what triggers a rollback rather than a fix-forward, so the decision is made calmly in advance.
4. Rehearse it once on staging. An unrehearsed runbook is a guess written down.
5. Keep it beside the restore procedure from section 5 and the credential rotation procedure from section 7.

FILES

`operations runbook`

DONE WHEN

A rollback has been rehearsed on staging and timed, and the database position is explicit.

12. Monitoring and operations

7 requirements marked missing. Sentry, PostHog and a well-designed health probe already exist, which is more than most products have at this stage. What is missing is everything that turns that instrumentation into someone being told. Nothing currently polls the health endpoint, and no alert exists for any condition.

Uptime monitoring and alerting

task R7

WHY IT MATTERS

GET /api/health exists and is well designed: it reports status plus capability booleans for database, sharedRateLimit and ai, without touching the database or any paid API. Nothing polls it, and no alert fires for any condition, so an outage is discovered by a user.

STEPS

1. Point an external monitor at /api/health on a one-minute interval. Any hosted uptime service will do.
2. Alert on the status field, not just the HTTP code.
3. Assert the capability booleans too. sharedRateLimit false in production means rate limiting has silently degraded to per-instance memory, which is the section 7 risk and is otherwise invisible.
4. Add a second check against a real page, not only the API, so a broken render is caught.
5. Configure Sentry alerting: new issue types, and a spike in error rate.
6. Route alerts somewhere a human actually sees out of hours.

FILES

`external monitoring service, Sentry alert rules`

DONE WHEN

Stopping the production app produces an alert to a human within two minutes, and a degraded rate limiter alerts without an outage.

Incident response runbook and escalation path

task R7

WHY IT MATTERS

No runbook and no on-call path exist. With a single operator this feels unnecessary, which is exactly why it gets skipped and then needed.

STEPS

1. Write the first version in one page. Who is called, how to reach the hosting and database dashboards, how to roll back (section 11), and how to restore (section 5).
2. List the three or four most likely failures and what to do for each: database unreachable, Stripe webhook failing, AI provider down, deploy broken.
3. State explicitly what counts as an incident worth waking someone for.
4. Include a user-communication line. For a paid product, silence during an outage is its own damage.
5. Keep it in the repository, beside the other runbooks, so it is versioned with the code it describes.

FILES

`operations runbook in the repository`

DONE WHEN

One page exists covering the likely failures, the rollback and restore paths, and who responds.

Structured request logging and retention

task R7

WHY IT MATTERS

Logging is console.error only. When something goes wrong in production you will have unstructured lines with no request correlation, which makes an incident much harder to reason about. No retention policy is stated, which is also a privacy gap given the policy work in section 6.

STEPS

1. Introduce a small logging helper emitting structured JSON: timestamp, level, route, status, duration and a request id.
2. Never log personal data. Resume content, email bodies, tokens and passwords must never reach a log line. This is both a privacy and a security requirement.
3. Correlate with Sentry by including the same request id, so an error and its request can be matched.
4. Set and document a retention period, and make sure it matches what the privacy policy claims.
5. Replace the existing console.error calls progressively rather than in one sweep, to avoid the wide-blast-radius conflict problem described in section 11.

FILES

`new helper in src/services, API route handlers`

DONE WHEN

Logs are structured and correlatable, contain no personal data, and have a documented retention period that matches the privacy policy.

Status page

task optional

WHY IT MATTERS

None exists. For a single-operator product at launch this is reasonably deferred, but it is where users will look during an outage.

STEPS

1. Defer for launch and record the decision.
2. In the meantime, decide where you would communicate an outage, and make sure the contact route from section 1 can carry that message.
3. Revisit once you have paying customers who depend on availability.

FILES

`none for launch`

DONE WHEN

A recorded decision, with a communication channel identified even if no status page exists.

After this guide

Every MISSING requirement in the inventory is covered above. That is the engineering half of launch readiness. Three things remain, and none of them is in this document.

1. Check the sixteen blocked requirements

Production environment variables and domains, the live database schema and its backups, Stripe dashboard configuration including live-versus-test mode and the webhook endpoint, the Resend sending domain's SPF, DKIM and DMARC records, and repository branch protection. Each can block a launch on its own, and none announces itself. Record what you find rather than assuming it is fine.

2. Run the journeys against staging

Signup, login, build and download, checkout, entitlement revocation and deletion, against a staging deployment with its own database and Stripe test keys. Then prove the destructive paths actually work: restore a backup, revoke a subscription, delete an account, and confirm each did what it claimed.

3. Verify the live site after deploying

A green deploy is not a verified product. Re-run the critical journeys against production, confirm the webhook receives real events, and watch the monitoring you set up in section 12 for the first day rather than assuming silence means success.

A NOTE ON COMPLETENESS

This guide addresses what a source review found. A static review only finds what it thinks to look for. The concurrent-save data-loss bug in section 5 was not found by reading the code looking for bugs - it surfaced because an existing QA handbook asked a question the audit had not thought to ask. One prompt from one document produced one real defect, and there is no reason to believe it was the last. Expect the end-to-end tests, the staging pass and the first real users to each surface things this guide does not contain.